

Nombre: Andy Paul Sánchez Pilaloa

Curso: 7mo Software “A”

Asignatura: Aplicaciones Distribuidas

Fecha: Martes, 2 de junio de 2026

Título: Implementación de un prototipo de RegistroDistribuido con sockets TCP, Lamport, heartbeats, Bully y autenticación por token

Repositorio: <https://github.com/AndySanchez2004/Aplicaciones-Distribuidas>

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Materiales y métodos	2
3.1. Entorno de desarrollo	2
3.2. Creación del proyecto	3
4. Desarrollo de la implementación	4
4.1. Parte A — Comunicación entre procesos	4
4.2. Parte B — Reloj lógico de Lamport	5
4.3. Parte C — Heartbeats y detección de fallos	6
4.4. Parte D — Coordinación con Bully	7
4.5. Parte E — Seguridad por token	8
5. Resultados obtenidos	9
5.1. Ejecución de nodos	9
5.2. Cliente autenticado y operaciones	10
5.3. Detección de fallos	11
6. Análisis técnico	12
6.1. Compromiso CAP	12
6.2. Falacias de la computación distribuida consideradas	13
6.3. Transparencias ofrecidas	13
6.4. SLA propuesto	13
6.5. Comparación con Raft	14
7. Conclusiones	14
8. Anexos	15
8.1. Código completo de Cliente.java	15
8.2. Código completo de NodoDistribuido.java	15

1. Introducción

En esta práctica se desarrolló un prototipo llamado *RegistroDistribuido*, con el objetivo de aplicar varios conceptos vistos en la Unidad 1 de Aplicaciones Distribuidas. El sistema fue implementado en Java usando sockets TCP y se ejecutó de forma local en IntelliJ IDEA mediante tres nodos: Nodo 1, Nodo 2 y Nodo 3, asociados a los puertos 9001, 9002 y 9003.

El prototipo permite que un cliente se conecte a un nodo, se autentique mediante un token y envíe operaciones tipo **MSG**. Además, cada nodo mantiene un contador lógico de Lamport, envía heartbeats a los demás nodos y detecta cuando alguno deja de responder. También se incluyó una aproximación al algoritmo Bully para representar la elección de coordinador.

La finalidad principal fue construir una demostración funcional, pequeña y entendible, donde se evidencien comunicación entre procesos, sincronización lógica, tolerancia a fallos, coordinación y seguridad básica.

2. Objetivos

2.1. Objetivo general

Implementar un prototipo distribuido en Java que demuestre comunicación TCP, reloj lógico de Lamport, heartbeats, detección de fallos, coordinación básica con Bully y autenticación por token.

2.2. Objetivos específicos

- Crear un cliente TCP capaz de conectarse a un nodo y enviar operaciones.
- Ejecutar tres nodos locales en puertos diferentes.
- Validar un token antes de aceptar la comunicación del cliente.
- Incrementar un reloj lógico cada vez que se recibe una operación.
- Enviar heartbeats entre nodos para detectar fallos.
- Registrar evidencias de ejecución mediante capturas de pantalla.

3. Materiales y métodos

3.1. Entorno de desarrollo

El prototipo fue realizado en Java desde IntelliJ IDEA, usando Maven como estructura base del proyecto. La ejecución se hizo localmente, utilizando diferentes puertos para simular varios nodos en una misma máquina.

Cuadro 1: Herramientas utilizadas

Elemento	Detalle
Lenguaje	Java
IDE	IntelliJ IDEA
Proyecto	Maven
Comunicación	Sockets TCP
Ejecución	Localhost
Puertos	9001, 9002 y 9003

3.2. Creación del proyecto

El proyecto se creó en IntelliJ IDEA desde la opción *File* → *New* → *Project*, seleccionando el arquetipo `maven-archetype-quickstart`. La configuración usada fue:

Cuadro 2: Configuración inicial del proyecto

Parámetro	Valor
Nombre	<code>servidor-tcp</code>
GroupId	<code>ec.edu.uteq.distribuidas</code>
ArtifactId	<code>servidor-tcp</code>
Arquetipo	<code>maven-archetype-quickstart</code>

Dentro del paquete principal se trabajó con dos clases principales: `Cliente.java`, encargada de conectarse y enviar operaciones, y `NodoDistribuido.java`, encargada de funcionar como nodo servidor, manejar mensajes, enviar heartbeats y detectar fallos.

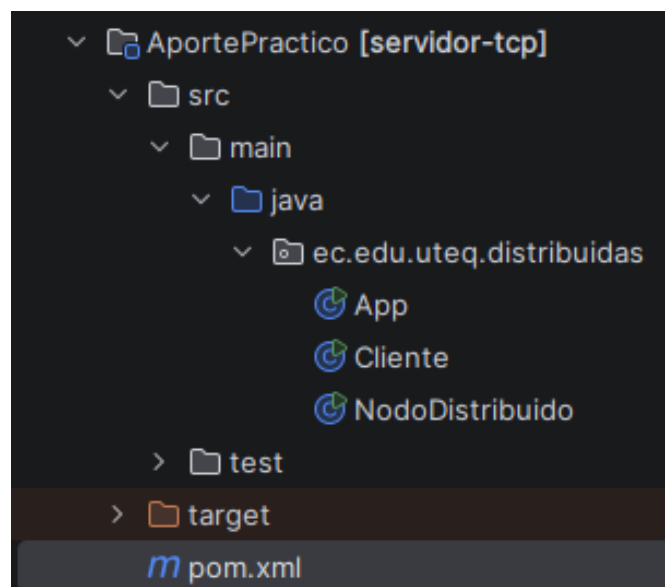


Figura 1: Estructura del proyecto en IntelliJ IDEA.

4. Desarrollo de la implementación

4.1. Parte A — Comunicación entre procesos

La comunicación se realizó con sockets TCP. El cliente se conecta al Nodo 1 mediante localhost y el puerto 9001. Para enviar mensajes se usó `PrintWriter`, y para recibir respuestas se usó `BufferedReader`.

```
1 Socket socket = new Socket("localhost", 9001);
2 BufferedReader in = new BufferedReader(
3     new InputStreamReader(socket.getInputStream()));
4 PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
5 Scanner sc = new Scanner(System.in);
```

Listing 1: Conexión TCP del cliente.

Los mensajes se delimitan con saltos de línea, usando `println()` para enviar y `readLine()` para recibir. Esto permite que cada mensaje sea procesado de forma individual.

```
1 while (true) {
2     System.out.print("> ");
3     String msg = sc.nextLine();
4
5     out.println(msg);
6     System.out.println(in.readLine());
7 }
```

Listing 2: Envío y recepción de mensajes desde el cliente.

En cada nodo se levanta un servidor TCP. Cuando llega una conexión, se crea un hilo para atenderla sin bloquear al resto del sistema.

```
1 private void server() {
2     try (ServerSocket ss = new ServerSocket(puerto)) {
3         while (true) {
4             Socket socket = ss.accept();
5             new Thread(() -> handle(socket)).start();
6         }
7     } catch (Exception e) {
8         e.printStackTrace();
9     }
10 }
```

Listing 3: Servidor TCP del nodo distribuido.

```
// ----- SERVER -----
private void server() { 1 usage  Anyod26
    try (ServerSocket ss = new ServerSocket(puerto)) {

        while (true) {
            Socket socket = ss.accept();
            new Thread(() -> handle(socket)).start();
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Figura 2: Inicio del nodo distribuido y servidor TCP.

4.2. Parte B — Reloj lógico de Lamport

Cada nodo tiene una variable llamada `relojLamport`. Cuando el nodo recibe un mensaje que empieza con `MSG`, incrementa el contador y responde con un `ACK` que incluye el valor actual del reloj.

```
1 if (msg.startsWith("MSG")) {
2     relojLamport++;
3     out.println("NODO " + id + " ACK Lamport=" + relojLamport);
4 }
```

Listing 4: Actualización del reloj lógico de Lamport.

En esta versión se aplicó el incremento local del reloj. Para una versión más completa, los mensajes entre nodos deberían incluir también la marca lógica del emisor y aplicar la regla `max(local, recibido) + 1`.

```
// LAMPORT MESSAGE
if (msg.startsWith("MSG")) {
    relojLamport++;
    out.println("NODO " + id + " ACK Lamport=" + relojLamport);
}
```

Figura 3: Procesamiento de operaciones con reloj Lamport.

4.3. Parte C — Heartbeats y detección de fallos

Cada nodo ejecuta una tarea periódica que intenta comunicarse con los demás nodos cada tres segundos. Si un nodo no responde, se registra como caído.

```
1 private void heartbeatTask() {
2     while (true) {
3         try {
4             Thread.sleep(3000);
5
6             for (int nodo : puertos.keySet()) {
7                 if (nodo != id) {
8                     send(nodo, "HB");
9                 }
10            }
11        } catch (Exception ignored) {}
12    }
13 }
```

Listing 5: Envío periódico de heartbeats.

La detección del fallo ocurre dentro del método `send`. Si la conexión TCP no se puede establecer, el nodo imprime en consola qué nodo fue detectado como caído.

```
1 catch (Exception e) {
2     System.out.println("Nodo " + nodo + " caído detectado por " +
3         id);
4     nodosActivos.remove(nodo);
5 }
```

Listing 6: Registro de nodo caído.

```
// ----- HEARTBEAT -----
private void heartbeatTask() {
    while (true) {
        try {
            Thread.sleep(3000);

            for (int nodo : puertos.keySet()) {
                if (nodo != id) {
                    send(nodo, "HB");
                }
            }
        } catch (Exception ignored) {}
    }
}
}
```

Figura 4: Heartbeat y detección de nodos caídos.

4.4. Parte D — Coordinación con Bully

El Nodo 3 se definió como coordinador inicial, porque tiene el identificador más alto. Esta idea sigue el principio del algoritmo Bully, donde el nodo activo con mayor ID debe actuar como líder.

```
1 if (id == 3) {
2     coordinador.set(true);
3     System.out.println("Nodo " + id + " es COORDINADOR inicial");
4 }
```

Listing 7: Coordinador inicial.

También se incluyó un método de elección para iniciar el proceso cuando se detecta la caída del coordinador.

```
1 private void startElection() {
2     System.out.println("Nodo " + id + " inicia ELECTION");
3
4     for (int nodo : nodosActivos) {
5         if (nodo > id) {
6             send(nodo, "ELECTION");
7         }
8     }
9 }
```

```

7         }
8     }
9
10    coordinador.set(true);
11    System.out.println("Nodo " + id + " es nuevo COORDINADOR");
12 }

```

Listing 8: Inicio de elección de coordinador.

La implementación representa una versión básica del algoritmo. En una versión más robusta, el nodo debería esperar respuestas OK antes de declararse coordinador.

```

// ----- BULLY -----
private void startElection() { 1 usage  Anyod26

    System.out.println("Nodo " + id + " inicia ELECTION");

    for (int nodo : nodosActivos) {
        if (nodo > id) {
            send(nodo, msg: "ELECTION");
        }
    }

    coordinador.set(true);
    System.out.println("Nodo " + id + " es nuevo COORDINADOR");
}

```

Figura 5: Método de elección de coordinador.

4.5. Parte E — Seguridad por token

Antes de enviar operaciones, el cliente manda el token TOKEN:123. El nodo lo valida y responde OK AUTH si es correcto o TOKEN INVALIDO si no coincide.

```

1 if (msg.startsWith("TOKEN:")) {
2     if (!msg.split(":")[1].equals("123")) {
3         out.println("TOKEN INVALIDO");
4         continue;
5     }
6     out.println("OK AUTH");
7     continue;
8 }

```

Listing 9: Validación del token.

Este mecanismo no reemplaza una seguridad real con cifrado o sesiones, pero sí permite demostrar una autenticación básica antes de aceptar operaciones.

5. Resultados obtenidos

5.1. Ejecución de nodos

Se ejecutaron tres instancias de `NodoDistribuido`. Cada una usó un identificador y puerto diferente.

Cuadro 3: Nodos ejecutados

Nodo	ID	Puerto
Nodo 1	1	9001
Nodo 2	2	9002
Nodo 3	3	9003

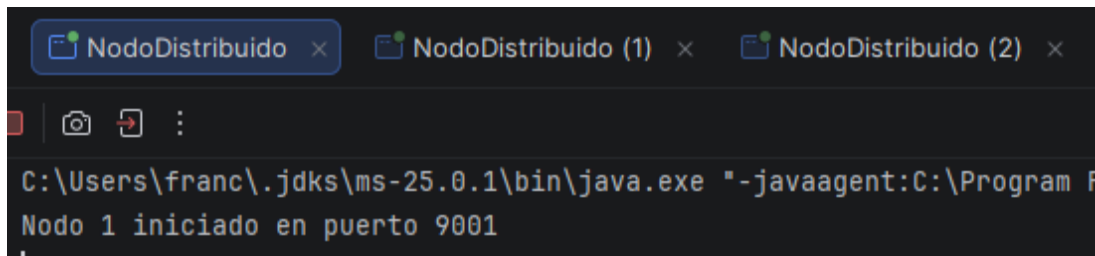


Figura 6: Nodo 1 iniciado en el puerto 9001.

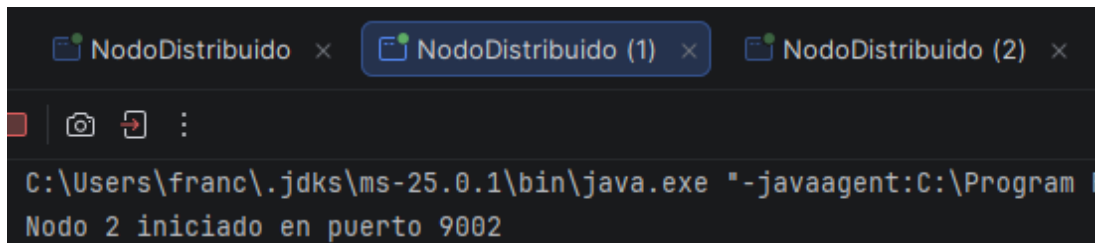


Figura 7: Nodo 2 iniciado en el puerto 9002.

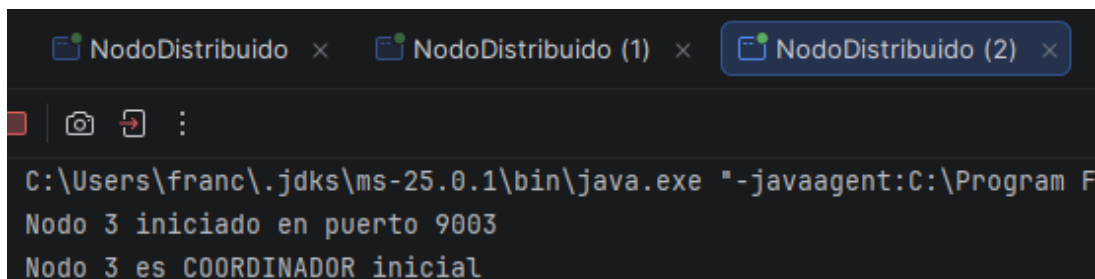
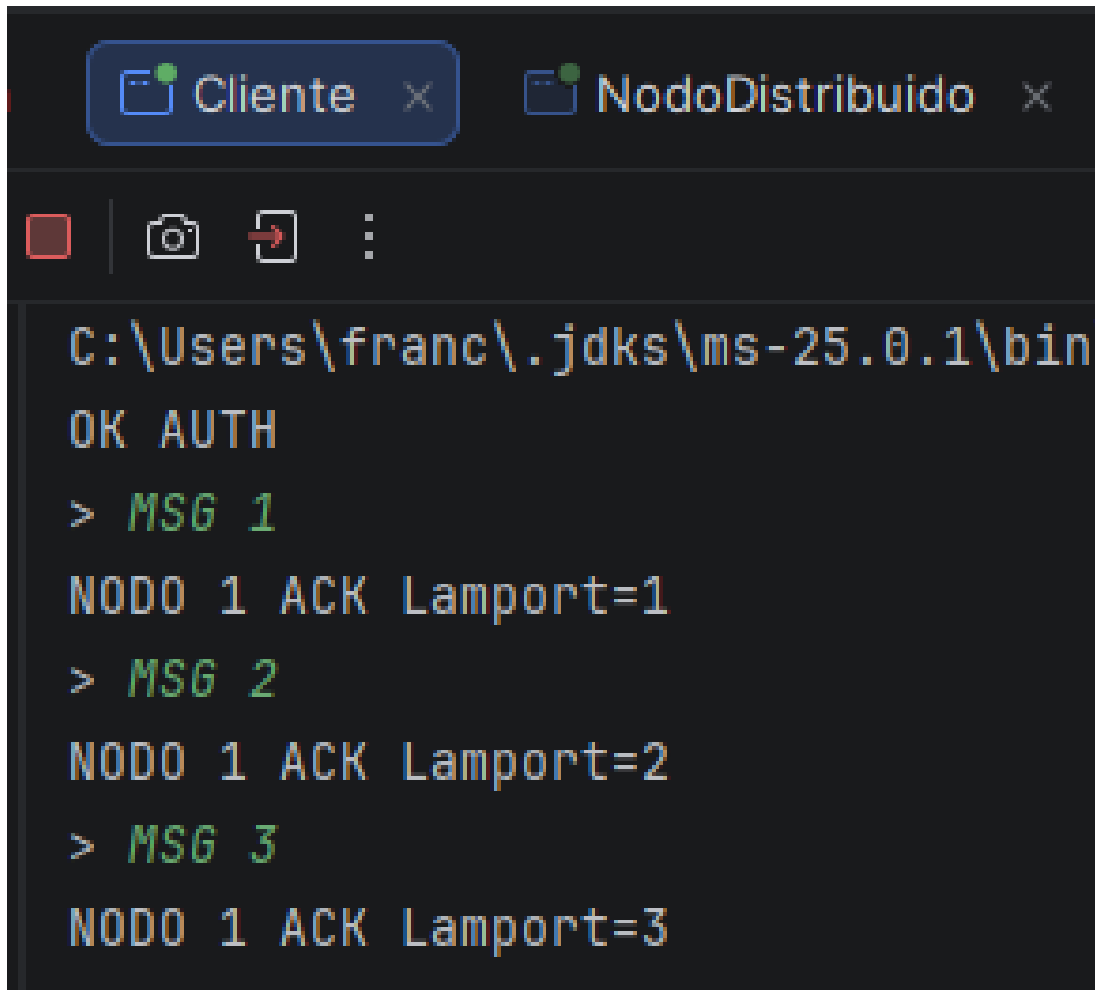


Figura 8: Nodo 3 iniciado como coordinador inicial.

Como se observa, los tres nodos iniciaron correctamente. El Nodo 3 asumió el rol de coordinador inicial.

5.2. Cliente autenticado y operaciones

El cliente se conectó al Nodo 1, envió el token y recibió la respuesta OK AUTH. Luego se enviaron operaciones tipo MSG, recibiendo respuestas con el valor del reloj Lamport.



```
C:\Users\franc\.jdk\ms-25.0.1\bin
OK AUTH
> MSG 1
NODO 1 ACK Lamport=1
> MSG 2
NODO 1 ACK Lamport=2
> MSG 3
NODO 1 ACK Lamport=3
```

Figura 9: Cliente autenticado y operaciones con Lamport.

También se registró una prueba adicional de comportamiento con fallo durante la comunicación del cliente.

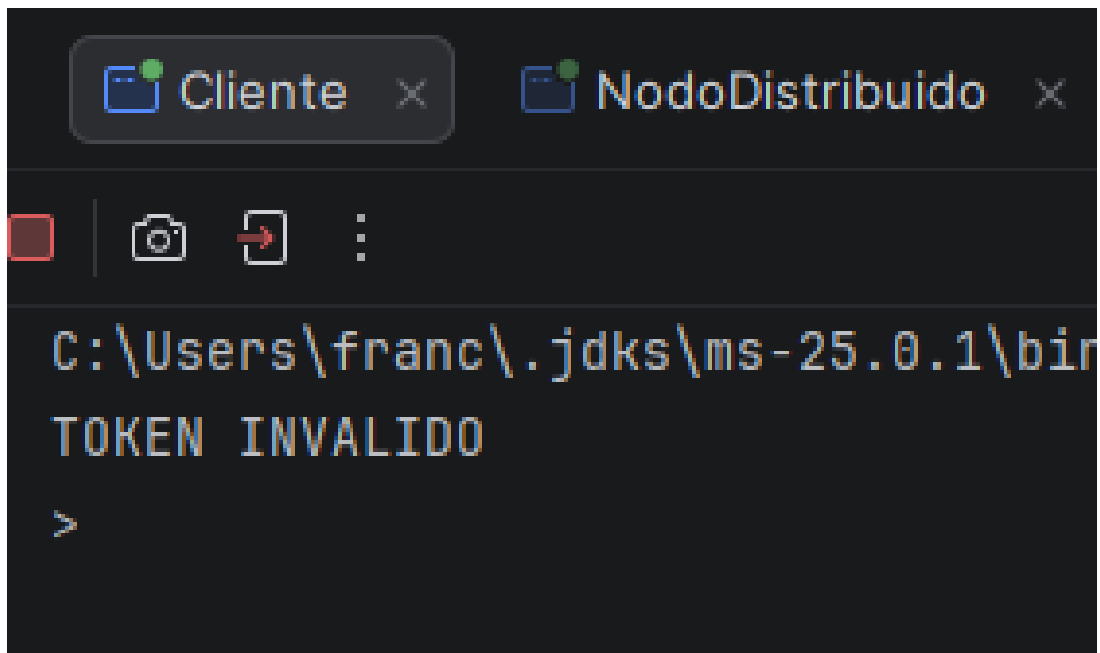


Figura 10: Evidencia adicional de comunicación del cliente durante la prueba.

5.3. Detección de fallos

Los nodos registraron en consola cuando otro nodo dejó de responder. Esta evidencia confirma que el mecanismo de heartbeats permitió detectar fallos parciales.

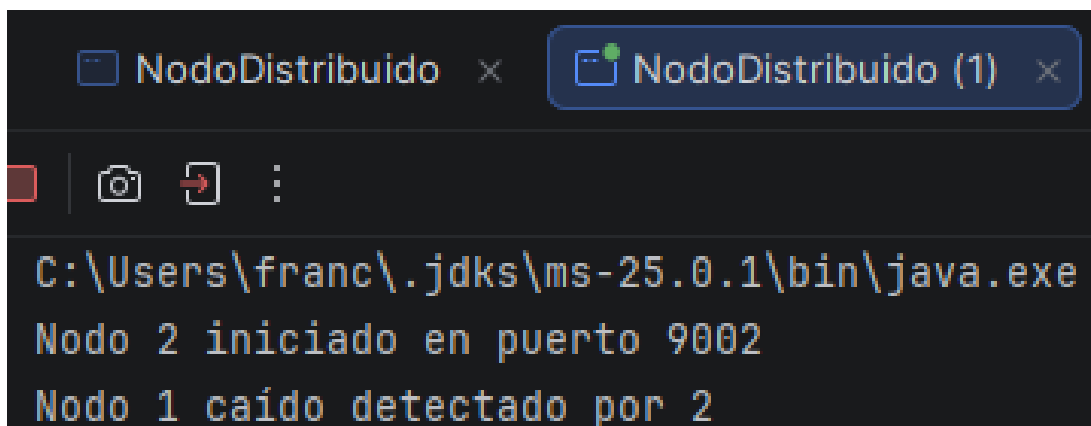
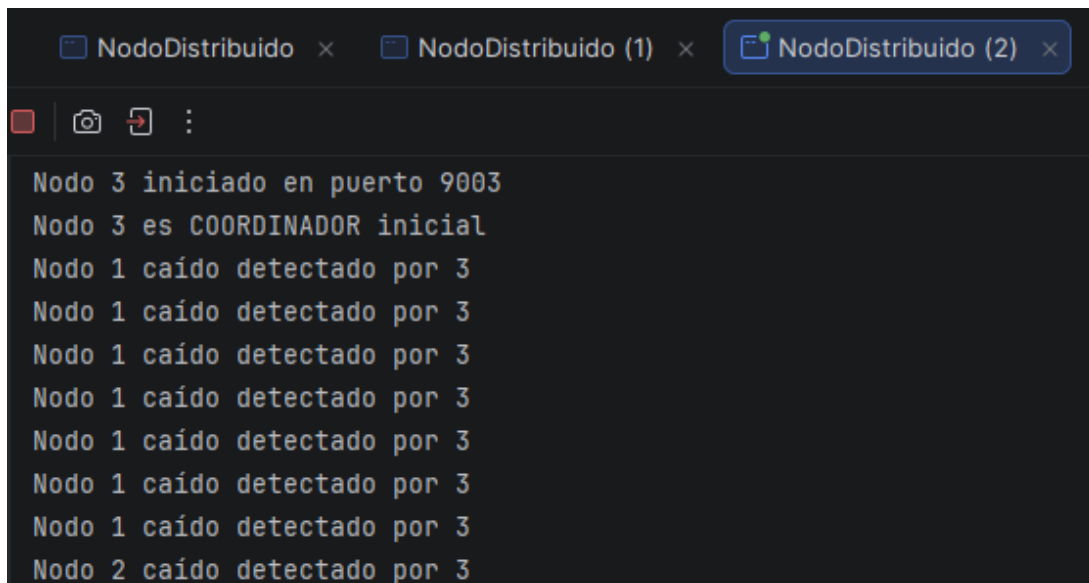


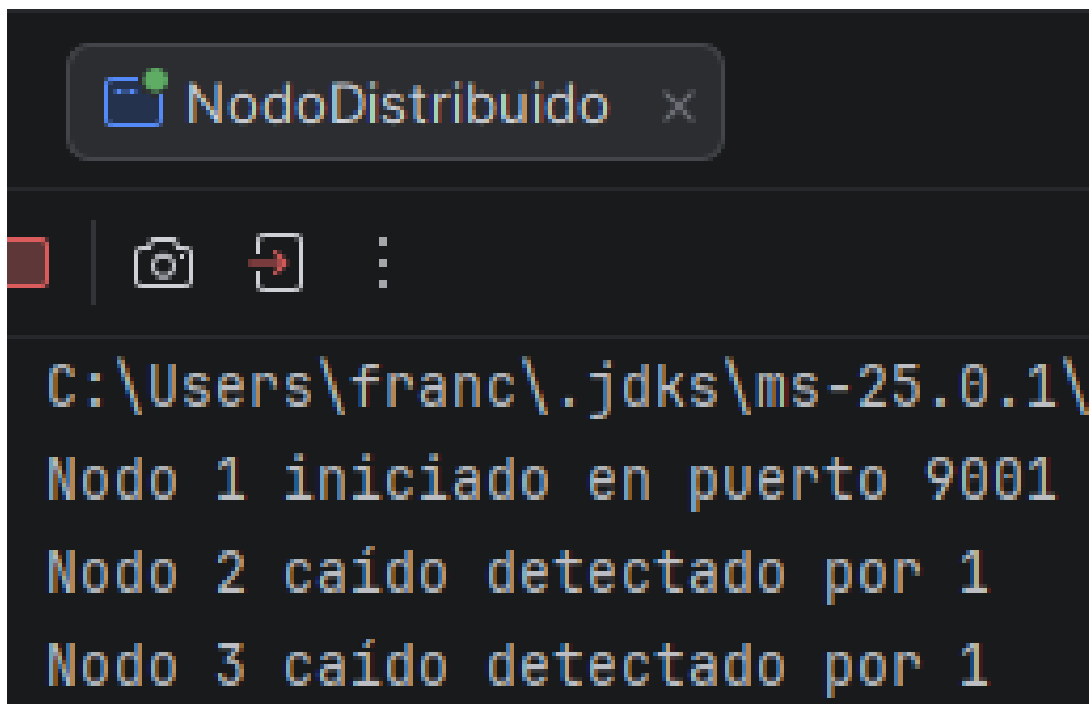
Figura 11: Detección de fallos desde el Nodo 1.



The screenshot shows a terminal window with three tabs: 'NodoDistribuido', 'NodoDistribuido (1)', and 'NodoDistribuido (2)'. The active tab is 'NodoDistribuido (2)'. The terminal output shows the following messages:

```
Nodo 3 iniciado en puerto 9003
Nodo 3 es COORDINADOR inicial
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 1 caído detectado por 3
Nodo 2 caído detectado por 3
```

Figura 12: Detección de fallos desde el Nodo 2.



The screenshot shows a terminal window with one tab: 'NodoDistribuido'. The terminal output shows the following messages:

```
C:\Users\franc\.jdk\ms-25.0.1\
Nodo 1 iniciado en puerto 9001
Nodo 2 caído detectado por 1
Nodo 3 caído detectado por 1
```

Figura 13: Detección de fallos desde el Nodo 3.

6. Análisis técnico

6.1. Compromiso CAP

El prototipo prioriza la disponibilidad y la tolerancia a fallos parciales. Cuando un nodo deja de responder, los demás continúan activos y registran la caída en consola. Sin embargo, no se garantiza consistencia fuerte, porque las operaciones no se replican en todos los nodos ni existe un consenso antes de responder al cliente.

Por esta razón, el comportamiento se acerca más a un enfoque AP: el sistema intenta seguir funcionando aunque existan fallos de comunicación. Para lograr un modelo CP sería necesario bloquear operaciones o usar un algoritmo de consenso antes de confirmar cada registro.

6.2. Falacias de la computación distribuida consideradas

Durante la práctica se evidenció que la red no siempre es confiable, ya que un nodo puede dejar de responder y generar una excepción de conexión. También se consideró que la latencia no es cero, por eso los heartbeats se ejecutan cada cierto intervalo y no de forma instantánea.

Otra falacia considerada fue asumir que la red es segura. Para evitar que cualquier cliente envíe operaciones sin control, se agregó una autenticación básica mediante token. Además, como TCP trabaja como flujo de bytes, los mensajes se delimitaron con saltos de línea usando `println()` y `readLine()`.

6.3. Transparencias ofrecidas

La solución ofrece una transparencia de acceso parcial, porque el cliente solo envía mensajes y recibe respuestas sin conocer todo el procesamiento interno del nodo. También existe una transparencia de fallos limitada, ya que los nodos detectan caídas mediante heartbeats.

No se ofrece transparencia de ubicación completa, porque el cliente conoce directamente el puerto 9001. Tampoco existe transparencia de replicación completa, ya que las operaciones no se copian automáticamente entre todos los nodos.

6.4. SLA propuesto

Para este prototipo se propone un SLA de disponibilidad del 99.9%. Esto permite calcular el tiempo máximo de inactividad anual con la siguiente fórmula:

$$T_{cada} = (1 - D) \times T_{total}$$

Donde:

$$D = 0,999$$

$$T_{total} = 8760horas/ao$$

Entonces:

$$T_{cada} = (1 - 0,999) \times 8760 = 8,76horas/ao$$

Por tanto, con un SLA del 99.9%, el sistema podría estar inactivo aproximadamente 8.76 horas al año. Si el SLA fuera de 99.99%, el tiempo de caída bajaría a aproximadamente 52.6 minutos anuales.

6.5. Comparación con Raft

Si se reemplazara Bully por Raft, el sistema tendría una elección de líder más robusta y podría mantener un log replicado entre nodos. Esto ayudaría a mejorar la consistencia de las operaciones.

El costo sería una mayor complejidad, más mensajes entre nodos y mayor latencia antes de confirmar una operación. Para esta práctica, Bully fue suficiente para representar la coordinación básica; para un sistema real, Raft sería más adecuado.

7. Conclusiones

Se logró implementar un prototipo distribuido en Java usando sockets TCP, con un cliente y tres nodos ejecutados localmente en los puertos 9001, 9002 y 9003. La comunicación funcionó mediante mensajes delimitados por salto de línea, permitiendo enviar operaciones y recibir respuestas desde consola.

También se incorporó autenticación básica por token. El cliente envía `TOKEN:123` y el nodo responde `OK AUTH` cuando la validación es correcta. Esto permitió demostrar un control inicial antes de aceptar operaciones.

El reloj lógico de Lamport se aplicó mediante un contador que aumenta cada vez que el nodo recibe una operación `MSG`. Aunque la implementación puede mejorar incorporando la regla completa `max(local, recibido) + 1`, el prototipo permitió observar el avance lógico de los eventos.

Finalmente, los heartbeats permitieron detectar nodos caídos durante la ejecución. El Nodo 3 fue definido como coordinador inicial y se agregó una versión básica del algoritmo Bully para representar la elección de líder.

8. Anexos

8.1. Código completo de Cliente.java

```
1 package ec.edu.uteq.distribuidas;
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.Scanner;
6
7 public class Cliente {
8
9     public static void main(String[] args) {
10
11         try (
12             Socket socket = new Socket("localhost", 9001);
13             BufferedReader in = new BufferedReader(new
14                 InputStreamReader(socket.getInputStream()));
15             PrintWriter out = new PrintWriter(socket.
16                 getOutputStream(), true);
17             Scanner sc = new Scanner(System.in)
18         ) {
19
20             // AUTH
21             out.println("TOKEN:123");
22             System.out.println(in.readLine());
23
24             while (true) {
25
26                 System.out.print("> ");
27                 String msg = sc.nextLine();
28
29                 out.println(msg);
30
31                 System.out.println(in.readLine());
32             }
33         } catch (Exception e) {
34             e.printStackTrace();
35         }
36 }
```

Listing 10: Código completo de Cliente.java

8.2. Código completo de NodoDistribuido.java

```
1 package ec.edu.uteq.distribuidas;
2
3 import java.io.*;
4 import java.net.*;
```

```

5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.concurrent.atomic.AtomicBoolean;
8
9 public class NodoDistribuido {
10
11     private final int id;
12     private final int puerto;
13
14     private int relojLamport = 0;
15
16     private final Set<Integer> nodosActivos = new HashSet<>();
17     private final Map<Integer, Integer> puertos = new HashMap<>();
18
19     private final AtomicBoolean coordinador = new AtomicBoolean(
20         false);
21     private volatile boolean activo = true;
22
23     public NodoDistribuido(int id, int puerto) {
24         this.id = id;
25         this.puerto = puerto;
26
27         puertos.put(1, 9001);
28         puertos.put(2, 9002);
29         puertos.put(3, 9003);
30
31         nodosActivos.add(1);
32         nodosActivos.add(2);
33         nodosActivos.add(3);
34     }
35
36     public static void main(String[] args) {
37         if (args.length < 2) {
38             System.err.println("Error: Faltan argumentos.");
39             System.err.println("Uso: java NodoDistribuido <id_nodo>
40                 <puerto>");
41             System.exit(1);
42         }
43
44         int id = Integer.parseInt(args[0]);
45         int puerto = Integer.parseInt(args[1]);
46
47         new NodoDistribuido(id, puerto).start();
48     }
49
50     public void start() {
51         System.out.println("Nodo " + id + " iniciado en puerto " +
52             puerto);
53
54         Executors.newSingleThreadExecutor().submit(this::
55             heartbeatTask);
56     }
57 }

```

```

52     Executors.newSingleThreadExecutor().submit(this::server);
53
54
55     if (id == 3) {
56         coordinador.set(true);
57         System.out.println("Nodo " + id + " es COORDINADOR
58             inicial");
59     }
60
61     private void server() {
62         try (ServerSocket ss = new ServerSocket(puerto)) {
63
64             while (true) {
65                 Socket socket = ss.accept();
66                 new Thread(() -> handle(socket)).start();
67             }
68
69             } catch (Exception e) {
70                 e.printStackTrace();
71             }
72         }
73
74     private void handle(Socket socket) {
75         try (
76             BufferedReader in = new BufferedReader(new
77                 InputStreamReader(socket.getInputStream()));
78             PrintWriter out = new PrintWriter(socket.
79                 getOutputStream(), true)
80         ) {
81
82             String msg;
83
84             while ((msg = in.readLine()) != null) {
85
86                 if (msg.startsWith("TOKEN:")) {
87                     if (!msg.split(":")[1].equals("123")) {
88                         out.println("TOKEN INVALIDO");
89                         continue;
90                     }
91                     out.println("OK AUTH");
92                     continue;
93                 }
94
95                 if (msg.startsWith("MSG")) {
96                     relojLamport++;
97                     out.println("NODO " + id + " ACK Lamport=" +
98                         relojLamport);
99                 }
100
101                 if (msg.equals("HB")) {

```

```

99         nodosActivos.add(1);
100     }
101
102     if (msg.equals("ELECTION")) {
103         if (id > 1) {
104             out.println("OK");
105         }
106     }
107
108     if (msg.equals("COORDINATOR")) {
109         coordinador.set(true);
110     }
111 }
112
113 } catch (Exception e) {
114     System.out.println("Nodo " + id + " ca do cliente");
115 }
116 }
117
118 private void heartbeatTask() {
119     while (true) {
120         try {
121             Thread.sleep(3000);
122
123             for (int nodo : puertos.keySet()) {
124                 if (nodo != id) {
125                     send(nodo, "HB");
126                 }
127             }
128
129             } catch (Exception ignored) {}
130     }
131 }
132
133 private void send(int nodo, String msg) {
134     try (Socket s = new Socket("localhost", puertos.get(nodo));
135         PrintWriter out = new PrintWriter(s.getOutputStream(),
136             true)) {
137
138         out.println(msg);
139
140     } catch (Exception e) {
141         System.out.println("Nodo " + nodo + " ca do detectado
142             por " + id);
143
144         nodosActivos.remove(nodo);
145
146         if (coordinador.get() && nodo == 3) {
147             startElection();
148         }
149     }
150 }

```

```

148     }
149
150     private void startElection() {
151
152         System.out.println("Nodo " + id + " inicia ELECTION");
153
154         for (int nodo : nodosActivos) {
155             if (nodo > id) {
156                 send(nodo, "ELECTION");
157             }
158         }
159
160         coordinador.set(true);
161         System.out.println("Nodo " + id + " es nuevo COORDINADOR");
162     }
163 }

```

Listing 11: Código completo de NodoDistribuido.java